Test Case 1

6. Railway Fare and Reservation Calculator Using Python

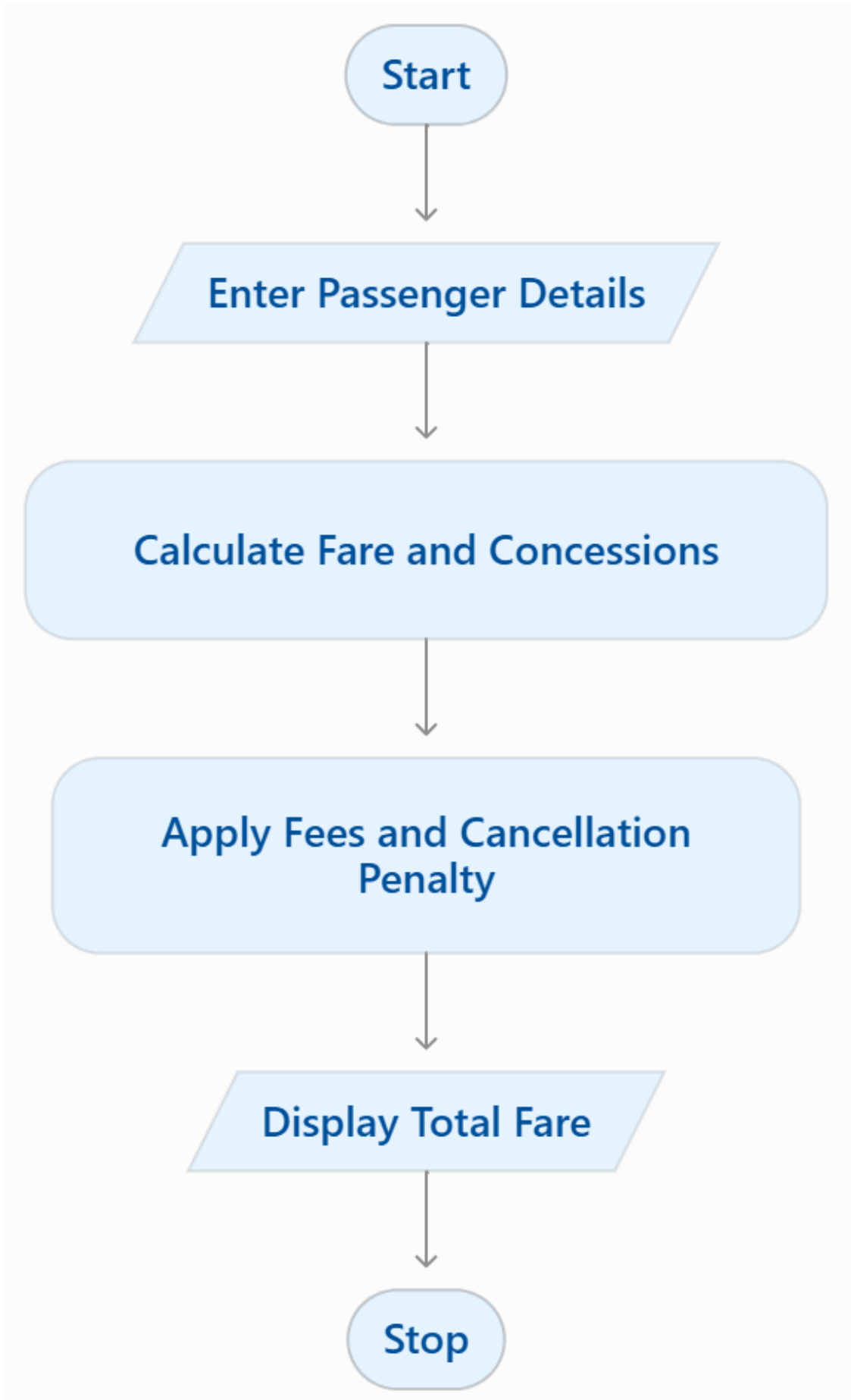
1. Problem Statement

Write a Python program to calculate railway fares for multiple passengers based on travel distance, travel class, age-based concessions, reservation fees, and cancellation penalties. The program should calculate the total fare and display the final payable amount.

2. Algorithm

1. Start.
2. Enter the number of passengers.
3. For each passenger, enter age, distance, travel class, and cancellation status.
4. Calculate the fare based on distance and class.
5. Apply the age-based concession.
6. Add the reservation fee.
7. Deduct the cancellation penalty for cancelled tickets.
8. Calculate and display the total payable amount.
9. Stop.

3. Flowchart


Start
Enter Passenger Details
Calculate Fare and Concessions
Apply Fees and Cancellation Penalty
Display Total Fare
Stop

4.Source Code

```
n = int(input("Enter number of passengers: "))
total = 0
if n <= 0:
    print("Invalid number of passengers.")
else:
    for i in range(n):
        print("\nPassenger", i + 1)
        age = int(input("Enter age: "))
        distance = float(input("Enter distance in km: "))
        travel_class = input("Enter class (sleeper/ac): ").lower()
        cancelled = input("Ticket cancelled? (yes/no): ").lower()
        if age < 0 or distance < 0 or travel_class not in ["sleeper", "ac"]:
            print("Invalid passenger details!")
            continue
        if travel_class == "sleeper":
            rate = 1.50
        else:
            rate = 3.00
        if age < 5:
            fare = 0
            reservation = 0
        else:
            fare = distance * rate
            if age >= 60:
                fare *= 0.80
            reservation = 50
        if cancelled == "yes":
```

```
            amount = max(0, fare + reservation - 100)
            print("Cancellation Penalty: Rs. 100")
        else:
            amount = fare + reservation
        print("Passenger Fare: Rs.", round(amount, 2))
        total += amount
    print("\nTotal Railway Fare: Rs.", round(total, 2))
```

5. Sample Input

```
Enter number of passengers: 2

Passenger 1
Enter age: 30
Enter distance in km: 200
Enter class (sleeper/ac): sleeper
Ticket cancelled? (yes/no): no

Passenger 2
Enter age: 65
Enter distance in km: 200
Enter class (sleeper/ac): ac
Ticket cancelled? (yes/no): yes
```

6. Sample Output

```
Passenger Fare: Rs. 350.0
Cancellation Penalty: Rs. 100
Passenger Fare: Rs. 430.0

Total Railway Fare: Rs. 780.0
```

Test Case 2

14. Warehouse Inventory Reconciliation

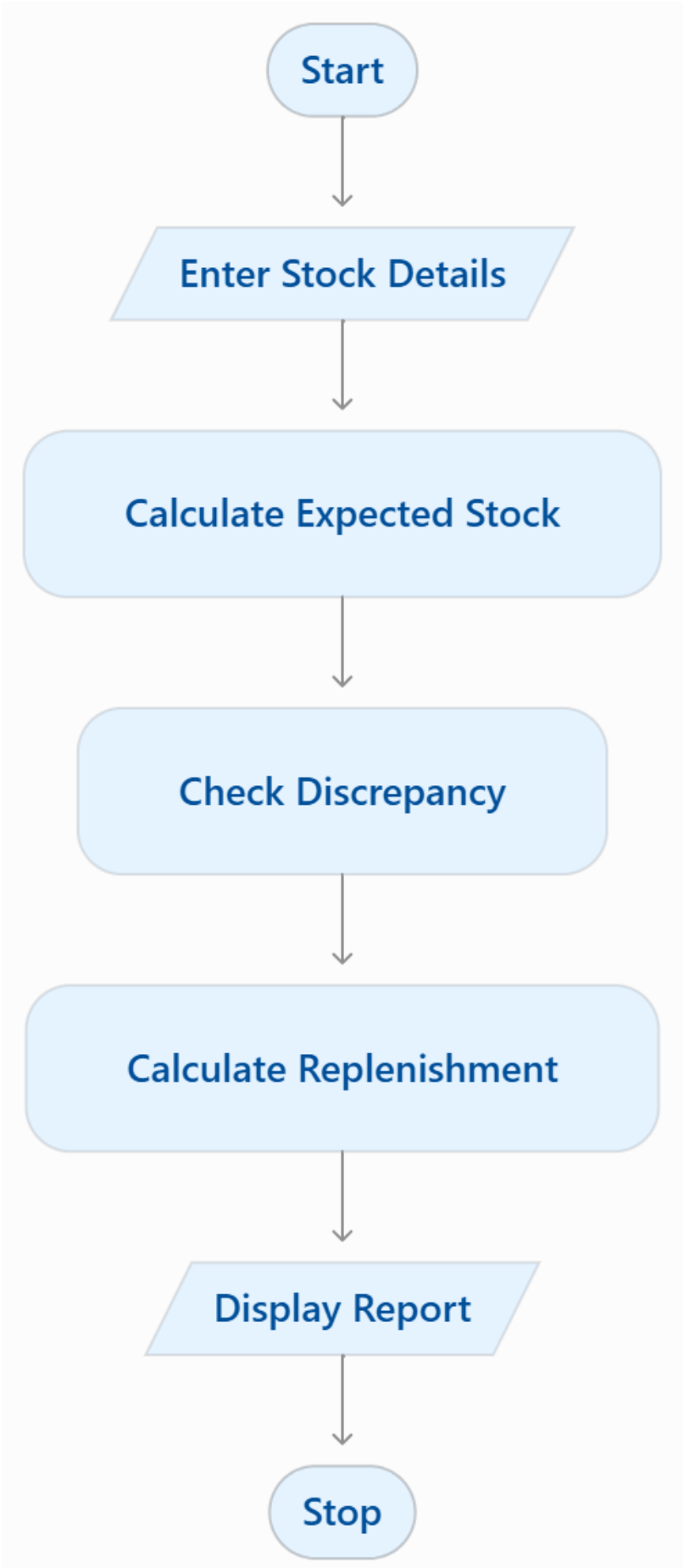
1. Problem Statement

Write a Python program to compare the opening stock, incoming quantities, dispatched quantities, and actual closing stock of multiple products. Detect stock discrepancies and calculate the additional quantity required for replenishment when stock falls below the minimum stock level.

2. Algorithm

1. Start the program.
2. Enter the number of products.
3. For each product, enter its name, opening stock, incoming quantity, dispatched quantity, actual closing stock, and minimum stock level.
4. Calculate expected closing stock using:
5. Compare expected closing stock with actual closing stock.
6. If both values differ, display the stock discrepancy.
7. If actual closing stock is below the minimum stock level, calculate the replenishment quantity.
8. Display the inventory report for all products.
9. Stop the program.

3. Flowchart


Start
Enter Stock Details
Calculate Expected Stock
Check Discrepancy
Calculate Replenishment
Display Report
Stop

4.Source Code

```
n = int(input("Enter number of products: "))
for i in range(n):
    print("\nProduct", i + 1)
    name = input("Enter product name: ")
    opening = int(input("Enter opening stock: "))
    incoming = int(input("Enter incoming quantity: "))
    dispatched = int(input("Enter dispatched quantity: "))
    actual = int(input("Enter actual closing stock: "))
    minimum = int(input("Enter minimum stock level: "))
    expected = opening + incoming - dispatched
    discrepancy = actual - expected
    replenishment = max(0, minimum - actual)
    print("\n--- Inventory Report ---")
    print("Product Name:", name)
    print("Expected Closing Stock:", expected)
    print("Actual Closing Stock:", actual)
    if discrepancy == 0:
        print("Stock Status: No Discrepancy")
    else:
        print("Stock Discrepancy:", discrepancy)
    print("Replenishment Required:", replenishment)
```

5. Sample Input

```
Enter number of products: 2

Product 1
Enter product name: Rice
Enter opening stock: 100
Enter incoming quantity: 50
Enter dispatched quantity: 80
Enter actual closing stock: 65
Enter minimum stock level: 40

Product 2
Enter product name: Sugar
Enter opening stock: 60
Enter incoming quantity: 20
Enter dispatched quantity: 50
Enter actual closing stock: 20
Enter minimum stock level: 40
```

6. Sample Output

```
--- Inventory Report ---
Product Name: Rice
Expected Closing Stock: 70
Actual Closing Stock: 65
Stock Discrepancy: -5
Replenishment Required: 0

--- Inventory Report ---
Product Name: Sugar
Expected Closing Stock: 30
Actual Closing Stock: 20
Stock Discrepancy: -10
Replenishment Required: 20
```

Test Case 3

19. Multi-Product Shopping Budget Constraint

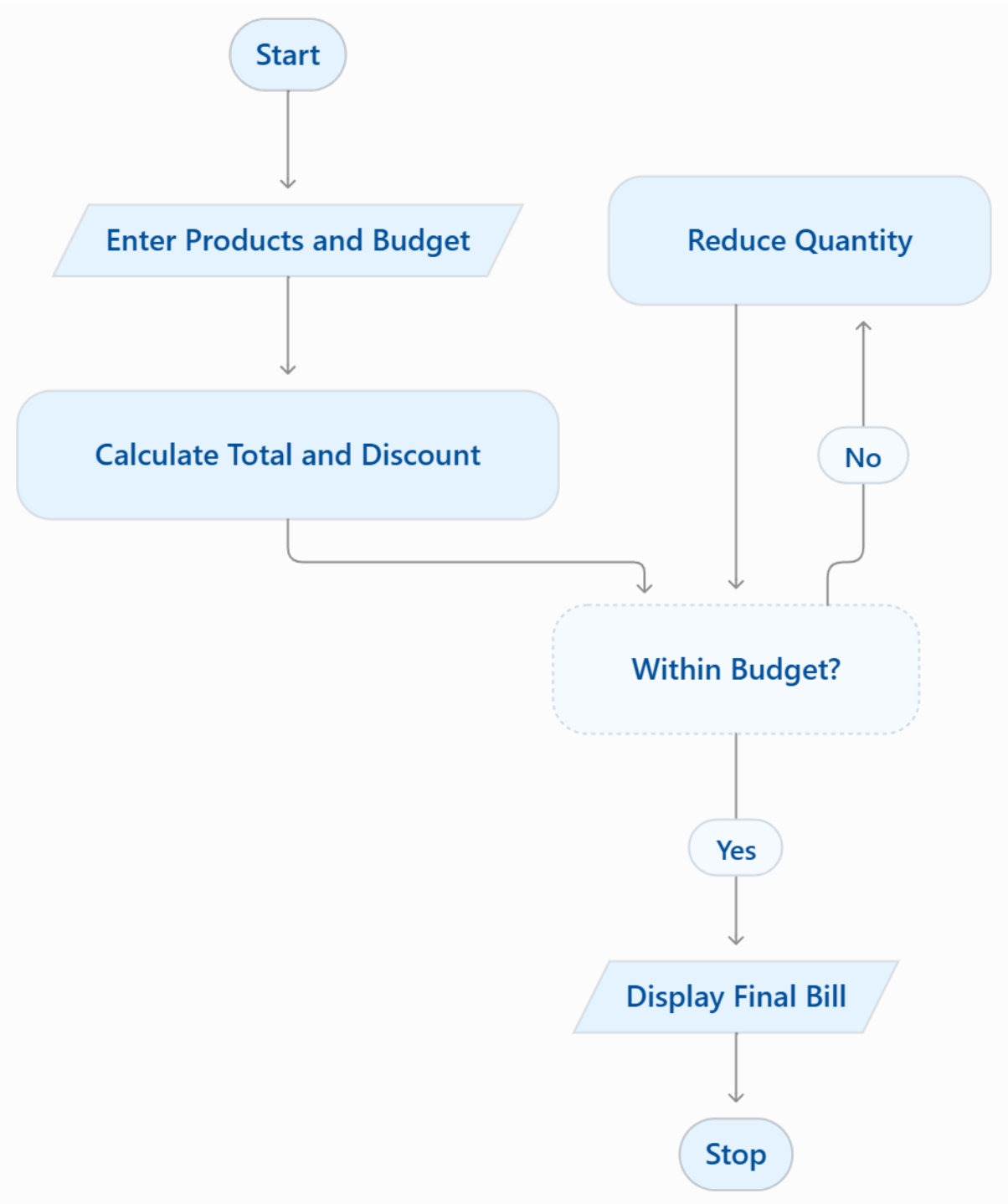
1. Problem Statement

Write a Python program to calculate the total cost of multiple products and quantities, apply a discount when the purchase amount reaches a specified limit, and reduce product quantities if the total cost exceeds the available budget.

2. Algorithm

1. Start the program.
2. Enter the shopping budget and number of products.
3. Enter each product name, price, and quantity.
4. Calculate the total shopping cost.
5. Apply a 10% discount if the total cost is ₹1,000 or more.
6. If the final cost exceeds the budget, reduce product quantities until the cost fits the budget.
7. Display the final quantities, discount, and total cost.
8. Stop the program.

3. Flowchart



4.Source Code

```
budget = float(input("Enter your budget: "))

n = int(input("Enter number of products: "))

products = []

for i in range(n):

    print("\nProduct", i + 1)
```

```
    name = input("Enter product name: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))
    products.append([name, price, quantity])
def calculate_total():
    total = sum(p[1] * p[2] for p in products)
    discount = total * 0.10 if total >= 1000 else 0
    return total - discount, discount
total, discount = calculate_total()
while total > budget:
    reduced = False
    for p in products:
        if p[2] > 0:
            p[2] -= 1
            reduced = True
            total, discount = calculate_total()
            if total <= budget:
                break
    if not reduced:
        break
print("\n--- Final Shopping Bill ---")
for p in products:
    print(p[0], "- Quantity:", p[2],
          "Cost: Rs.", round(p[1] * p[2], 2))
subtotal = sum(p[1] * p[2] for p in products)
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Final Total: Rs.", round(total, 2))
```

print("Remaining Budget: Rs.", round(budget - total, 2))

5. Sample Input

```
Enter your budget: 1000
Enter number of products: 3

Product 1
Enter product name: Shirt
Enter price: 400
Enter quantity: 2

Product 2
Enter product name: Shoes
Enter price: 600
Enter quantity: 1

Product 3
Enter product name: Cap
Enter price: 100
Enter quantity: 2
```

6. Sample Output

```
--- Final Shopping Bill ---
Shirt - Quantity: 1 Cost: Rs. 400.0
Shoes - Quantity: 0 Cost: Rs. 0.0
Cap - Quantity: 2 Cost: Rs. 200.0

Subtotal: Rs. 600.0
Discount: Rs. 0.0
Final Total: Rs. 600.0
Remaining Budget: Rs. 400.0
```

Test Case 4

24. Courier Charge Optimization

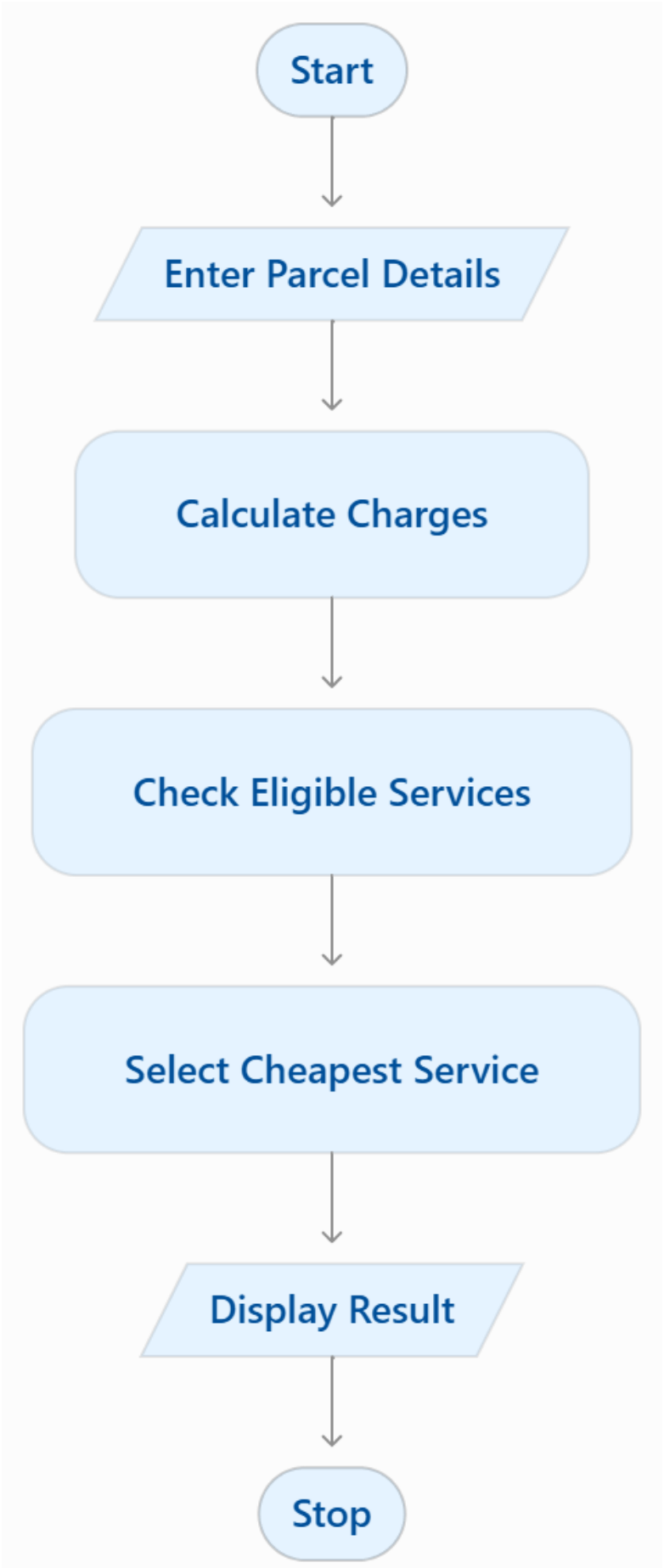
1. Problem Statement

Write a Python program to calculate courier charges based on parcel weight, dimensions, distance zone, urgency, and restrictions. Compare available courier services and select the cheapest eligible service.

2. Algorithm

1. Start the program.
2. Enter parcel weight, length, width, height, distance zone, urgency, and restricted status.
3. Calculate the parcel's volume.
4. Check which courier services are eligible.
5. Calculate charges based on weight, zone, urgency, and parcel volume.
6. Compare the charges of eligible services.
7. Select and display the cheapest eligible service.
8. Stop the program.

3. Flowchart


Start
Enter Parcel Details
Calculate Charges
Check Eligible Services
Select Cheapest Service
Display Result
Stop

4.Source Code

5. Sample Input

```
budget = float(input("Enter your budget: "))
n = int(input("Enter number of products: "))
products = []
for i in range(n):
    print("\nProduct", i + 1)
    name = input("Enter product name: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))
    products.append([name, price, quantity])
def calculate_total():
    total = sum(p[1] * p[2] for p in products)
    discount = total * 0.10 if total >= 1000 else 0
    return total - discount, discount
total, discount = calculate_total()
while total > budget:
    reduced = False
    for p in products:
        if p[2] > 0:
            p[2] -= 1
            reduced = True
            total, discount = calculate_total()
            if total <= budget:
                break
    if not reduced:
        break
print("\n--- Final Shopping Bill ---")
```

```
for p in products:
    print(p[0], "- Quantity:", p[2],
          "Cost: Rs.", round(p[1] * p[2], 2))
subtotal = sum(p[1] * p[2] for p in products)
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Final Total: Rs.", round(total, 2))
print("Remaining Budget: Rs.", round(budget - total, 2))
```

5.Sample Input

```
Enter parcel weight in kg: 5
Enter length in cm: 20
Enter width in cm: 15
Enter height in cm: 10
Enter distance zone (Local/National): Local
Enter urgency (Normal/Express): Normal
Is the parcel restricted? (yes/no): no
```

6. Sample Output

```
Eligible Courier Services:
Basic Courier - Rs. 220.0
Fast Courier - Rs. 430.0
Premium Courier - Rs. 520.0

Cheapest Service: Basic Courier
Total Courier Charge: Rs. 220.0
```

Test Case 5

35. Text Compression Using Character Runs

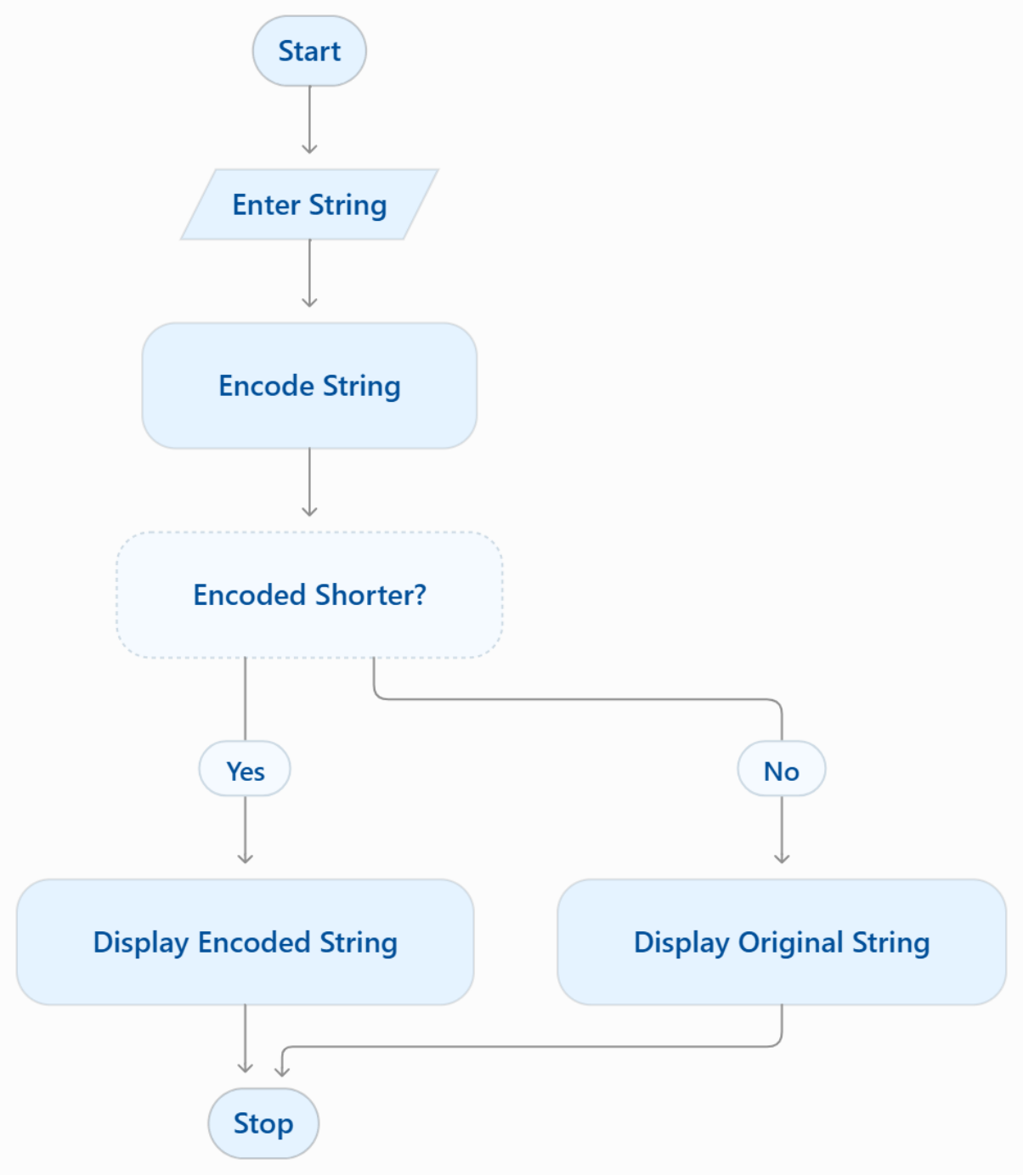
1. Problem Statement

Write a Python program to compress a string by grouping consecutive repeated characters. Compare the original string length with the encoded string length and display the encoded string only when compression is beneficial.

2. Algorithm

1. Start the program.
2. Enter a string.
3. Count consecutive repeated characters.
4. Encode each group using the character followed by its count.
5. Compare the original and encoded lengths.
6. If the encoded string is shorter, display it; otherwise, display the original string.
7. Stop the program.

3. Flowchart



4.Source Code

```
text = input("Enter a string: ")

encoded = ""

i = 0

while i < len(text):

    count = 1

    while i + 1 < len(text) and text[i] == text[i + 1]:
```

```
        count += 1
        i += 1
    encoded += text[i] + str(count)
    i += 1
print("Original String:", text)
print("Original Length:", len(text))
print("Encoded String:", encoded)
print("Encoded Length:", len(encoded))
if len(encoded) < len(text):
    print("Compression is beneficial.")
else:
    print("Compression is not beneficial.")
    print("Final String:", text)
```

5. Sample Input

```
Enter a string: aaabbccccccc
```

6. Sample Output

```
Original String: aaabbccccccc
Original Length: 11
Encoded String: a3b2c6
Encoded Length: 6
Compression is beneficial.
```

Test Case 6

45. Weekly Electricity Consumption Matrix

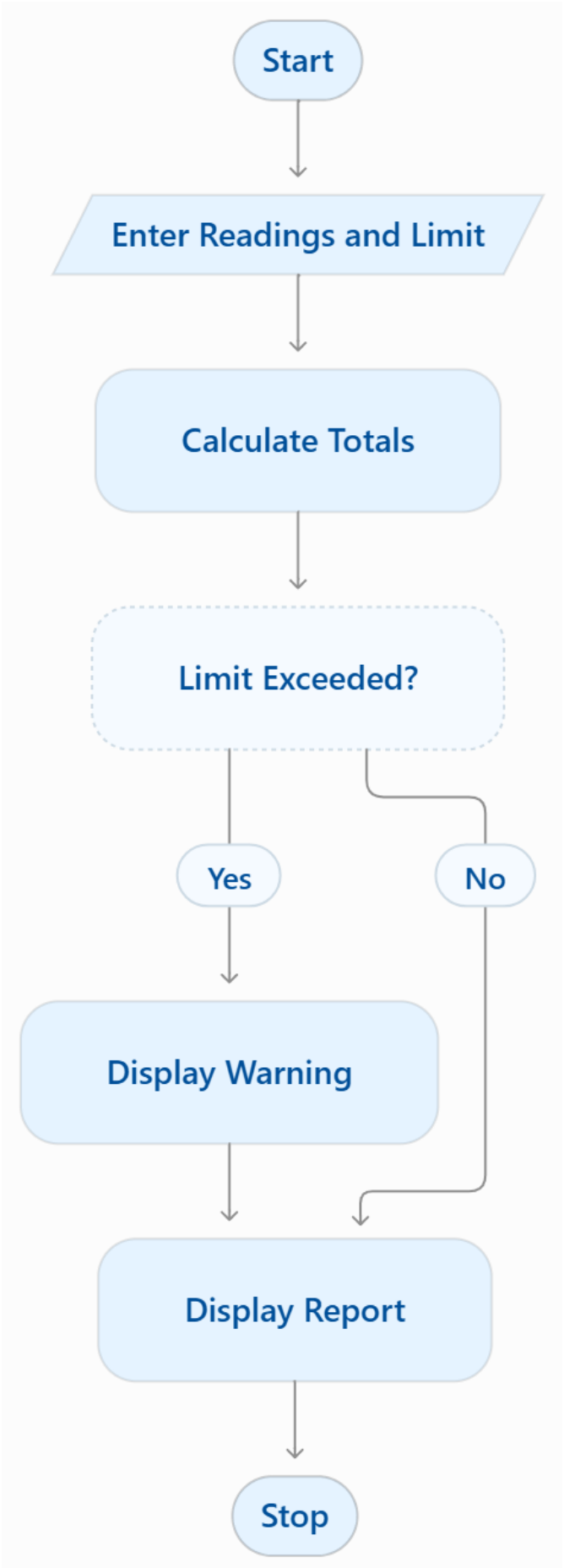
1. Problem Statement

Write a Python program to store electricity consumption readings for different appliances over seven days. Calculate daily and appliance-wise total consumption and identify days when electricity usage exceeds a specified limit.

2. Algorithm

1. Start the program.
2. Enter the number of appliances and the daily consumption limit.
3. Enter the electricity readings for each appliance for seven days.
4. Calculate the total consumption of each appliance.
5. Calculate the total consumption for each day.
6. Compare daily consumption with the specified limit.
7. Display appliance totals, daily totals, and excess consumption alerts.
8. Stop the program.

3. Flowchart


Start
Enter Readings and Limit
Calculate Totals
Limit Exceeded?
Yes
No
Display Warning
Display Report
Stop

4.Source Code

```
appliances = ["Fan", "AC", "Refrigerator"]
days = 7
limit = float(input("Enter daily consumption limit in kWh: "))
readings = []
for appliance in appliances:
    print("\nEnter 7-day readings for", appliance)
    values = []
    for day in range(days):
        value = float(input("Day " + str(day + 1) + ": "))
        values.append(value)
    readings.append(values)
print("\n--- Appliance-wise Total ---")
for i in range(len(appliances)):
    print(appliances[i], ":", sum(readings[i]), "kWh")
print("\n--- Daily Consumption ---")
for day in range(days):
    total = sum(readings[i][day] for i in range(len(appliances)))
    print("Day", day + 1, ":", total, "kWh")
    if total > limit:
        print("Warning: Consumption exceeds limit!")
```

5. Sample Input

```
Day 5: 2
Day 6: 2
Day 7: 2

Enter 7-day readings for AC
Day 1: 5
Day 2: 6
Day 3: 4
Day 4: 5
Day 5: 6
Day 6: 4
Day 7: 5

Enter 7-day readings for Refrigerator
Day 1: 2
Day 2: 2
Day 3: 2
Day 4: 2
Day 5: 2
Day 6: 2
Day 7: 2
```

6. Sample Output

```
--- Appliance-wise Total ---
Fan : 14.0 kWh
AC : 35.0 kWh
Refrigerator : 14.0 kWh

--- Daily Consumption ---
Day 1 : 9.0 kWh
Day 2 : 10.0 kWh
Day 3 : 8.0 kWh
Day 4 : 9.0 kWh
Day 5 : 10.0 kWh
Day 6 : 8.0 kWh
Day 7 : 9.0 kWh
```

Test Case 7

52. Hackathon Team Formation and Ranking

1. Problem Statement

Write a Python program to form hackathon teams, validate team sizes, calculate total scores from multiple rounds, and rank teams based on their scores. If two teams have the same total score, use the highest score in the final round to decide the ranking.

2. Algorithm

1. Start the program.
2. Enter the number of teams and the minimum and maximum team sizes.
3. Enter each team's name, member count, and scores for three rounds.
4. Validate the team size.
5. Calculate each team's total score.
6. Sort teams by total score in descending order.
7. If totals are tied, compare final-round scores.
8. Display valid teams and their rankings.
9. Stop the program.

3. Flowchart


Start
Enter Team Details
Validate Team Size
Calculate Scores
Rank Teams and Resolve Ties
Display Rankings
Stop

4.Source Code

```
teams = []
n = int(input("Enter number of teams: "))
min_size = 2
max_size = 4
for i in range(n):
    print("\nTeam", i + 1)
    name = input("Enter team name: ")
    members = int(input("Enter number of members: "))
    if members < min_size or members > max_size:
        print("Invalid team size! Team rejected.")
        continue
    r1 = int(input("Enter Round 1 score: "))
    r2 = int(input("Enter Round 2 score: "))
    r3 = int(input("Enter Round 3 score: "))
    total = r1 + r2 + r3
    teams.append({
        "name": name,
        "members": members,
        "total": total,
        "final": r3
    })
teams.sort(key=lambda t: (t["total"], t["final"]), reverse=True)
print("\n--- Hackathon Rankings ---")
for rank, team in enumerate(teams, start=1):
    print(rank, team["name"],
          "| Members:", team["members"],
          "| Total Score:", team["total"],
```

"| Final Round:", team["final"])

5. Sample Input

```
Enter number of teams: 3

Team 1
Enter team name: Alpha
Enter number of members: 3
Enter Round 1 score: 80
Enter Round 2 score: 85
Enter Round 3 score: 90

Team 2
Enter team name: Beta
Enter number of members: 2
Enter Round 1 score: 85
Enter Round 2 score: 85
Enter Round 3 score: 95

Team 3
Enter team name: Gamma
Enter number of members: 5
```

6. Sample Output

```
Invalid team size! Team rejected.

--- Hackathon Rankings ---
1 Beta | Members: 2 | Total Score: 265 | Final Round: 95
2 Alpha | Members: 3 | Total Score: 255 | Final Round: 90
```

Test Case 8

79. Integrated Warehouse Order Prioritization

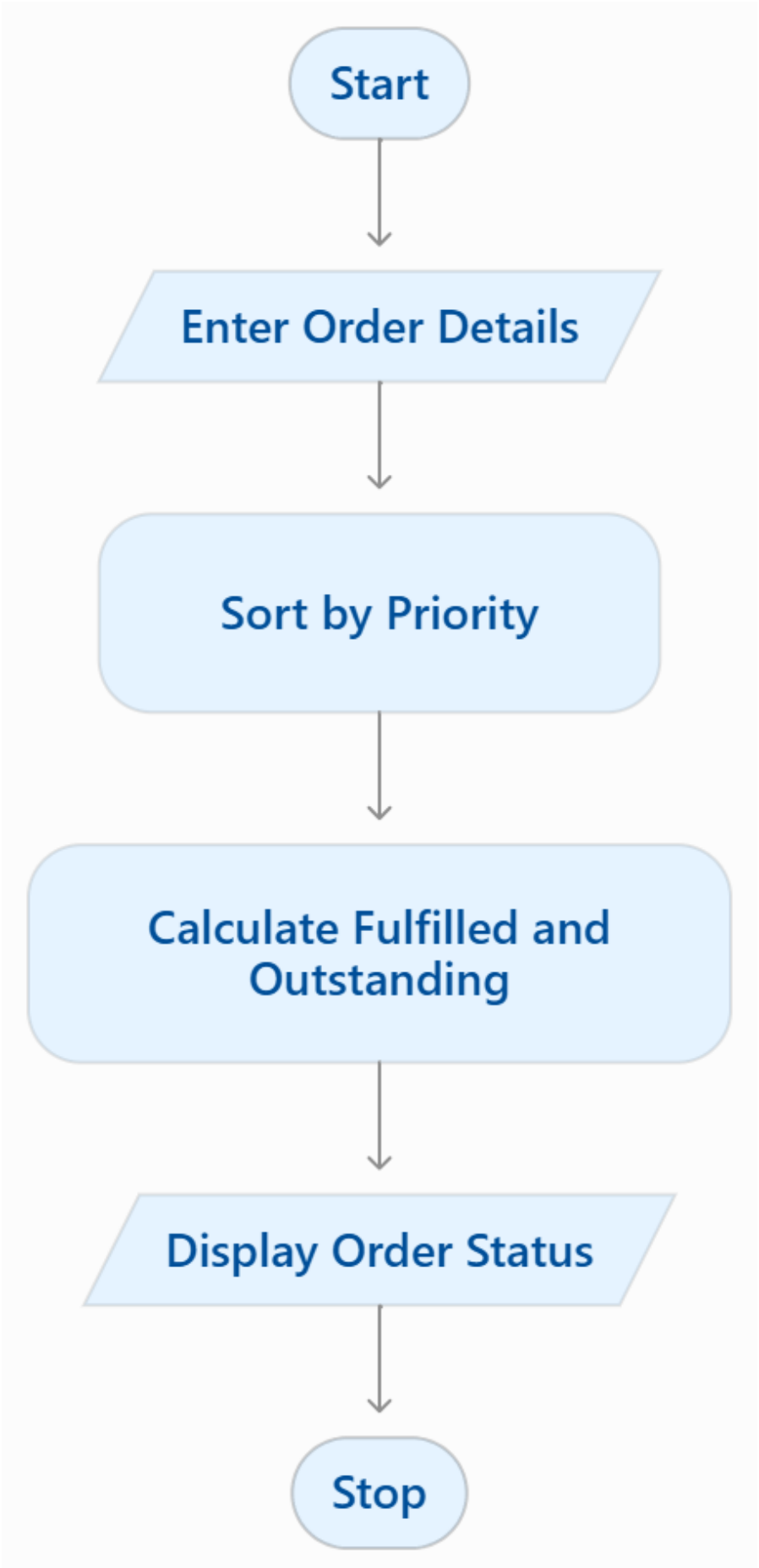
1. Problem Statement

Write a Python program to process customer orders based on product demand, available stock, order quantity, and priority category. Determine whether each order can be fulfilled and calculate the outstanding quantity for incomplete orders.

2. Algorithm

1. Start the program.
2. Enter the number of orders.
3. Enter product name, available stock, order quantity, and priority.
4. Sort orders by priority: High, Medium, and Low.
5. Compare available stock with the order quantity.
6. Calculate fulfilled and outstanding quantities.
7. Update stock after fulfilling each order.
8. Display the order status and outstanding quantity.
9. Stop the program.

3. Flowchart


Start
Enter Order Details
Sort by Priority
Calculate Fulfilled and Outstanding
Display Order Status
Stop

4.Source Code

```
orders = []
n = int(input("Enter number of orders: "))
for i in range(n):
    print("\nOrder", i + 1)
    name = input("Enter product name: ")
    stock = int(input("Enter available stock: "))
    quantity = int(input("Enter order quantity: "))
    priority = input("Enter priority (High/Medium/Low): ").title()
    orders.append({
        "name": name,
        "stock": stock,
        "quantity": quantity,
        "priority": priority
    })
priority_order = {"High": 1, "Medium": 2, "Low": 3}
orders.sort(key=lambda x: priority_order.get(x["priority"], 4))
stock_data = {}
for order in orders:
    name = order["name"]
    if name not in stock_data:
        stock_data[name] = order["stock"]
    available = stock_data[name]
    fulfilled = min(available, order["quantity"])
    outstanding = order["quantity"] - fulfilled
    stock_data[name] -= fulfilled
    if outstanding == 0:
        status = "Fulfilled"
```

```
    elif fulfilled > 0:
        status = "Partially Fulfilled"
    else:
        status = "Not Fulfilled"
    print("\nProduct:", name)
    print("Priority:", order["priority"])
    print("Fulfilled Quantity:", fulfilled)
    print("Outstanding Quantity:", outstanding)
    print("Status:", status)
```

5. Sample Input

```
Order 1
Enter product name: Rice
Enter available stock: 50
Enter order quantity: 30
Enter priority (High/Medium/Low): Low

Order 2
Enter product name: Rice
Enter available stock: 50
Enter order quantity: 25
Enter priority (High/Medium/Low): High

Order 3
Enter product name: Sugar
Enter available stock: 20
Enter order quantity: 15
Enter priority (High/Medium/Low): Medium
```

6. Sample Output

```
Product: Rice
Priority: High
Fulfilled Quantity: 25
Outstanding Quantity: 0
Status: Fulfilled

Product: Sugar
Priority: Medium
Fulfilled Quantity: 15
Outstanding Quantity: 0
Status: Fulfilled

Product: Rice
Priority: Low
Fulfilled Quantity: 25
Outstanding Quantity: 5
Status: Partially Fulfilled
```

Test Case 9

88. Multi-Branch Retail Inventory Transfer

1. Problem Statement

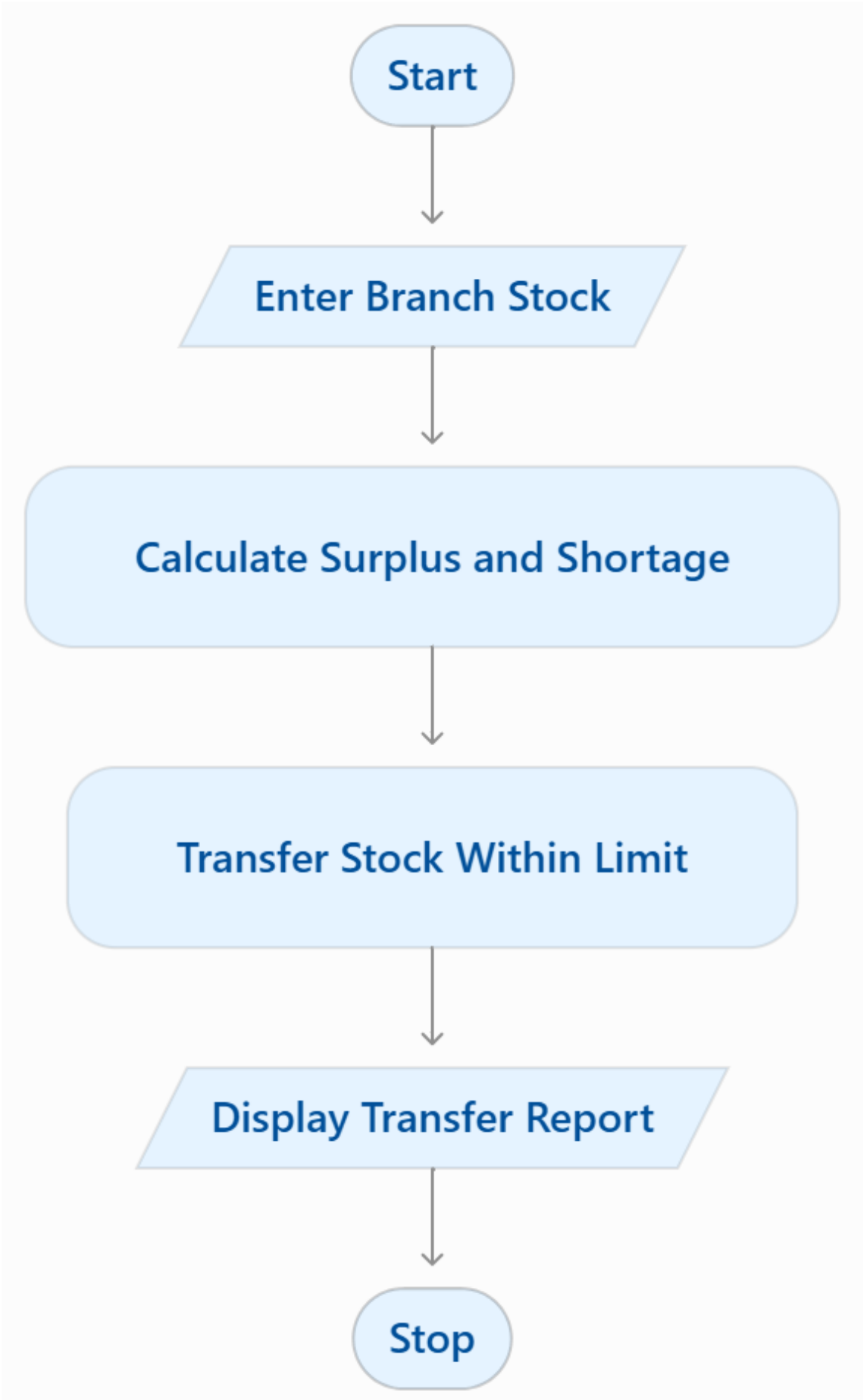
Write a Python program to compare stock available in different retail branches with their target stock levels. Calculate surpluses and shortages, and transfer products from surplus branches to shortage branches while respecting a maximum transfer quantity.

2. Algorithm

1. Start the program.
2. Enter the number of branches and products.
3. Enter each branch's current stock and target stock.
4. Calculate surplus and shortage for each branch.
5. Enter the maximum transfer quantity allowed.

6. Transfer stock from surplus branches to shortage branches.
7. Display the transfers and remaining shortages.
8. Stop the program.

3. Flowchart


Start
Enter Branch Stock
Calculate Surplus and Shortage
Transfer Stock Within Limit
Display Transfer Report
Stop

4.source Code

```
n = int(input("Enter number of branches: "))
branches = []
for i in range(n):
    print("\nBranch", i + 1)
    name = input("Enter branch name: ")
    stock = int(input("Enter current stock: "))
    target = int(input("Enter target stock: "))
    branches.append({
        "name": name,
        "stock": stock,
        "target": target,
        "surplus": max(0, stock - target),
        "shortage": max(0, target - stock)
    })
limit = int(input("\nEnter maximum transfer quantity: "))
print("\n--- Inventory Transfers ---")
for source in branches:
    for dest in branches:
        if source["surplus"] > 0 and dest["shortage"] > 0:
            qty = min(source["surplus"], dest["shortage"], limit)
            print(source["name"], "->", dest["name"], ":", qty, "units")
            source["surplus"] -= qty
            dest["shortage"] -= qty
print("\n--- Final Inventory Status ---")
for branch in branches:
    print(branch["name"])
    print("Remaining Surplus:", branch["surplus"])
```

```
    print("Remaining Shortage:", branch["shortage"])
```

5. Sample Input

```
Enter number of branches: 3

Branch 1
Enter branch name: Branch A
Enter current stock: 100
Enter target stock: 60

Branch 2
Enter branch name: Branch B
Enter current stock: 30
Enter target stock: 70

Branch 3
Enter branch name: Branch C
Enter current stock: 50
Enter target stock: 50

Enter maximum transfer quantity: 25
```

6. Sample Output

```
--- Inventory Transfers ---
Branch A -> Branch B : 25 units
Branch A -> Branch B : 15 units

--- Final Inventory Status ---
Branch A
Remaining Surplus: 0
Remaining Shortage: 0
Branch B
Remaining Surplus: 0
Remaining Shortage: 0
Branch C
Remaining Surplus: 0
Remaining Shortage: 0
```

Test Case 10

98. University Academic Progression Audit

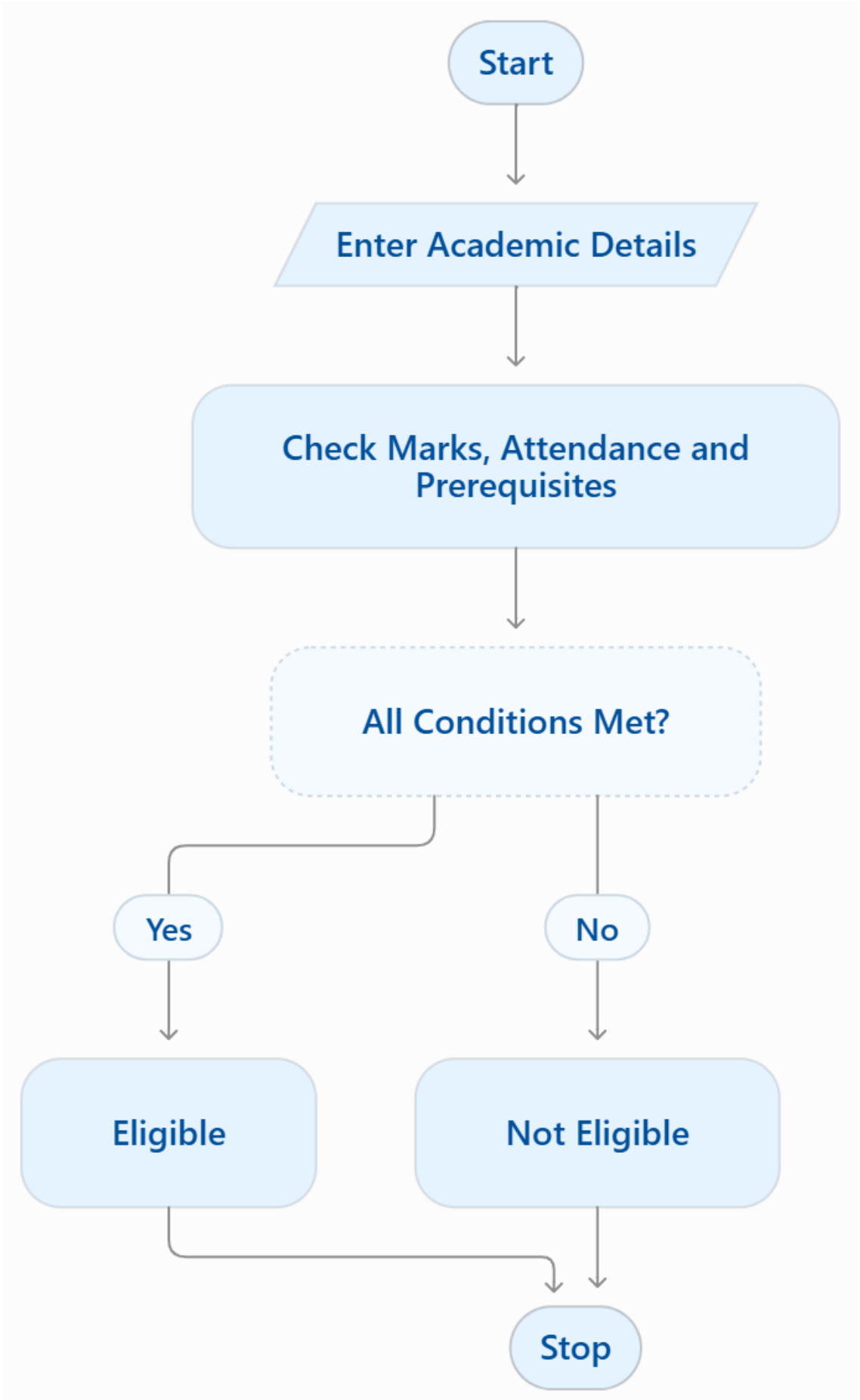
1. Problem Statement

Write a Python program to evaluate a student's academic progress using semester marks, subject pass conditions, attendance percentage, and prerequisite completion. Determine whether the student is eligible for progression and display any outstanding requirements.

2. Algorithm

1. Start the program.
2. Enter the number of subjects and marks for each subject.
3. Check whether each subject meets the minimum passing mark.
4. Enter attendance percentage and prerequisite completion status.
5. Check whether attendance is at least 75%.
6. Verify that all prerequisites are completed.
7. Display progression eligibility and outstanding requirements.
8. Stop the program.

3. Flowchart


Start
Enter Academic Details
Check Marks, Attendance and Prerequisites
All Conditions Met?
Yes
No
Eligible
Not Eligible
Stop

4.Source Code

```
n = int(input("Enter number of subjects: "))
pass_mark = 40
failed = []
for i in range(n):
    subject = input("Enter subject name: ")
    marks = float(input("Enter marks: "))
    if marks < pass_mark:
        failed.append(subject)
attendance = float(input("Enter attendance percentage: "))
prerequisites = input(
    "Are all prerequisites completed? (yes/no): "
).lower()
print("\n--- Academic Progression Report ---")
if failed:
    print("Failed Subjects:", ", ".join(failed))
else:
    print("All subjects passed.")
if attendance < 75:
    print("Outstanding Requirement: Attendance below 75%")
if prerequisites != "yes":
    print("Outstanding Requirement: Complete prerequisites")
if not failed and attendance >= 75 and prerequisites == "yes":
    print("Progression Status: Eligible")
else:
    print("Progression Status: Not Eligible")
```

5. Sample Input

```
Enter number of subjects: 3
Enter subject name: Mathematics
Enter marks: 75
Enter subject name: Python
Enter marks: 35
Enter subject name: English
Enter marks: 80
Enter attendance percentage: 70
Are all prerequisites completed? (yes/no): no
```

6. Sample Output

```
--- Academic Progression Report ---
Failed Subjects: Python
Outstanding Requirement: Attendance below 75%
Outstanding Requirement: Complete prerequisites
Progression Status: Not Eligible
```